A PLANT NURSERY PLATFORM

Growman

Commerce, plant care, and an internal operations dashboard, built as three clients on one Go API.

This document explains the working system end to end: the public website, the Expo mobile application, the admin dashboard, the Go backend, the data model, payments, inventory, shipping, privacy, and the answers a technical interview will press on.

Product	Growman, an Indian plant nursery store and care companion
Public site	growman.live
Clients	Next.js website, Expo mobile app, Next.js admin dashboard
System of record	Go HTTP API, PostgreSQL, Redis, Google Cloud Storage
Payments	Razorpay, with server-side pricing and stock reservation
Shipping	Shiprocket, with GST invoices on the Indian financial year
Audience	Final-year viva, technical interview, and onboarding

Website Next.js storefront on Cloudflare. Catalog, checkout, account, SEO.

Mobile Expo app. Same API, plus phone OTP, Truecaller, camera, care reminders.

Admin Staff dashboard. Orders, catalog edits, and a map of demand.

Contents

Read sections 1 to 4 for the product story, sections 5 to 11 before a systems interview, and section 20 the evening before the viva.

1. What this document is 5

What is in scope . . . 5

2. The product 5

What a customer can do . . . 5

What stays on the server . . . 6

Who the users are . . . 6

3. Problem, objectives, and scope 6

The problem . . . 6

Objectives . . . 6

What was deliberately not built . . . 6

4. System architecture 6

Runtime ports and production homes . . . 7

Why the website and the API are deployed differently . . . 7

Why there is no service layer . . . 7

Inside one API process . . . 7

Background workers . . . 8

5. Technology stack 8

Go dependencies that matter . . . 8

Monorepo tooling . . . 9

6. Repositories and code layout 9

Monorepo . . . 9

API boot order . . . 9

Mobile layout . . . 10

7. A request, end to end 10

Two exceptions . . . 10

Probes . . . 10

Route groups . . . 10

8. Data model 11

Catalog and commerce . . . 11

Care, privacy, and leads . . . 11

Three patterns that show up everywhere . . . 12

9. Authentication 12

Scope and role . . . 13

Where the token lives . . . 13

10. Catalog, search, cache, and images **13**

Two tiers . . . 13
Search . . . 14
Images . . . 14
What the website adds on top . . . 14

11. Checkout, stock, payments, tax, and shipping **14**

What create-order refuses to trust . . . 15
The database arbitrates . . . 15
Reserve, commit, release . . . 15
Fulfillment and GST . . . 16
Files to name . . . 16

12. The website **16**

How a page gets data . . . 16
Routes a customer actually uses . . . 16
SEO and the edge . . . 17
UI building blocks . . . 17

13. The mobile application **17**

Navigation . . . 17
Screens . . . 17
State . . . 18
What mobile has that the website does not . . . 18

14. The admin dashboard **19**

Screens . . . 19
The request map . . . 19
What the admin API can do beyond the current screens . . . 19

15. Dootha, plant care, and photo identification **20**

Dootha . . . 20
My plants . . . 20
Photo identification . . . 20
Care guides and the plant finder . . . 20

16. Privacy, security, and failure **20**

DPDP . . . 20
Security model . . . 20
Rate limits, fail open and fail closed . . . 21
When a dependency is down . . . 21

17. Deployment and local setup **21**

Production . . . 21
Environment names, not values . . . 21
Running it locally . . . 22

18. Testing, limits, and what changes at ten times the traffic 22

What is tested . . . 22
Honest weaknesses . . . 22
At ten times the traffic . . . 22

19. How a feature is added 23

API checklist . . . 23
Client checklist . . . 23
Questions before a change is finished . . . 23

20. Interview answers 23

21. File index 25

A closing summary you can speak . . . 26

1. What this document is

Growman is a working commerce platform for a plant nursery, with a public website, a native mobile application, and an internal admin dashboard. All three talk to one Go API. This document is the map of that system: what it does for a customer, how a request travels, where money and stock are decided, and how the three interfaces differ. It is written so you can explain the project in a final-year viva or a software interview without reading the repositories live.

The two codebases are `/home/venky/growman` (a pnpm and Turborepo monorepo) and `/home/venky/growman-mobile` (a separate Expo repository). The monorepo holds the website (`apps/web`), the admin dashboard (`apps/admin`), the API (`apps/go-backend`), and shared TypeScript packages. The mobile app is a sibling checkout on purpose: it needs a custom native binary, and it is not a workspace package of the web monorepo.

HOW TO USE THIS IN AN INTERVIEW

Open with the one-sentence architecture: three clients, one Go API, Postgres as the source of truth, Redis for cache and limits, Razorpay for money, Shiprocket for delivery. Then offer to draw checkout. If they ask where a feature lives, name the handler file and the screen file. If they ask what you would change, talk about type duplication, JWT storage, and moving background workers to a single leader before you talk about microservices.

What is in scope

In scope is the customer storefront on the web, the customer storefront on mobile, the staff dashboard, and the API that both of them trust. Out of scope is treating the admin app as a fourth product. Customers never see it. It is an internal client of the same API, gated by an admin role on the same user table.

2. The product

Growman sells plants and garden products for an Indian nursery, with the public site at growman.live and the physical context of a Hyderabad store. It is also a care companion after the purchase. A customer can browse a catalog, keep a cart on the device, check out as a guest or as a logged-in user, pay with Razorpay, track the order, download a GST invoice, and then keep the plant alive: watering, fertilizing, a care calendar, health notes, and a chat assistant named Dootha.

What a customer can do

The customer-facing product, on both website and mobile, covers these jobs:

- **Browse and search.** Categories, subcategories, brands, tags, sizes, and free-text search. The product page shows price, MRP, stock, sizes, reviews, and related plants.
- **Cart on the device.** The cart is not a database table. Web and mobile keep it in Zustand and persist it locally. Checkout posts the line items. The server looks up the current price and stock.
- **Checkout and pay.** Delivery quote, coupon, guest email verification, Razorpay checkout, order confirmation, retry of a failed payment, cancellation, and a GST invoice PDF.
- **Account.** Orders, wishlist, saved-for-later, addresses, payment-method labels, notifications, and a referral credit ledger.
- **Four ways to sign in.** Email and password, Google, Indian mobile OTP through MSG91, and Truecaller on Android. All of them become one User row and one JWT.
- **Dootha.** A plant-care chat. When the caller is logged in, the assistant can see that user's orders and wishlist. It does not place orders by itself.
- **Identify a plant from a photo** through Pl@ntNet, with a daily quota so a paid third-party call cannot run unbounded.
- **My plants.** A personal collection with water and fertilize actions, a health log, a care calendar, and local reminders on mobile.
- **Demand signals.** Request a missing plant, ask for a bulk quotation, join the Dootha waitlist.
- **Privacy under the DPDP Act.** Consent, a data export, and erasure that still respects a one-year floor on transactional records.

What stays on the server

Price, stock, coupons, tax, payment signatures, and who may see an order are server rules. The client may display a total. It may not decide one. A modified cart, a stale cached price, or a hand-edited request body cannot change what Razorpay is asked to charge. That sentence is the centre of the design, and it is the sentence to repeat if an interviewer asks why the cart is local.

Who the users are

Actor	Where they work	What they are allowed to do
Shopper, guest	Website or mobile	Browse, quote delivery, verify an email, place an order, look the order up with a public token
Shopper, account	Website or mobile	Everything a guest can do, plus wishlist, addresses, my plants, chat history, export and erase
Half-registered phone user	Mostly mobile	Only finish name and email. Scope is onboarding. They cannot place an order yet
Staff, admin or superadmin	Admin dashboard	Catalog writes, inventory, order operations, refunds, coupons, shipments, GST export, review moderation

Figure 1. Actors share one User table. Admin is a role, not a second identity system.

3. Problem, objectives, and scope

The problem

A nursery catalog is not a generic shop with a plant theme. Stock is physical and small: two people can try to buy the last specimen. Prices include GST. Delivery depends on pincode, distance from the store, and whether the customer will pick the plant up. Identity in India is often a mobile number, not an email. After the sale, the product is alive, so the relationship does not end at the invoice. A useful system has to get the money right, the stock right, and the care right, on a phone and in a browser, without trusting either client.

Objectives

- One catalog and one order history, reachable from a website and a mobile app.
- Checkout that cannot oversell, cannot double-charge, and cannot be repriced by the client.
- Indian payments, SMS OTP, GST invoices numbered on the April to March financial year, and courier booking.
- An internal dashboard for catalog edits, order operations, and a picture of where demand is coming from.
- A care layer (my plants, reminders, Dootha, photo identification) that uses the same account as the shop.
- Privacy controls that satisfy a right to erasure without deleting accounting records inside the legal retention floor.
- A deployment split that matches the traffic: edge-cached HTML for the site, a long-lived container for the API.

What was deliberately not built

There is no separate cart service, no message queue, and no microservice per domain. Background work (abandoned-stock release, shipment booking, retention) runs inside the API process. There is no generated TypeScript client: web and mobile copy JSON field names from the Go structs by hand. There is no second user table for staff. These are choices, and section 15 says when each one should be revisited.

4. System architecture

The shape is a read-heavy shop in front of a single writer. Browsing dominates. Orders, reviews, and profile edits are the writes. Traffic is concentrated in India, with daily and seasonal patterns, so the design optimises catalog reads and keeps a single relational primary. It does not try to be multi-region active-active.

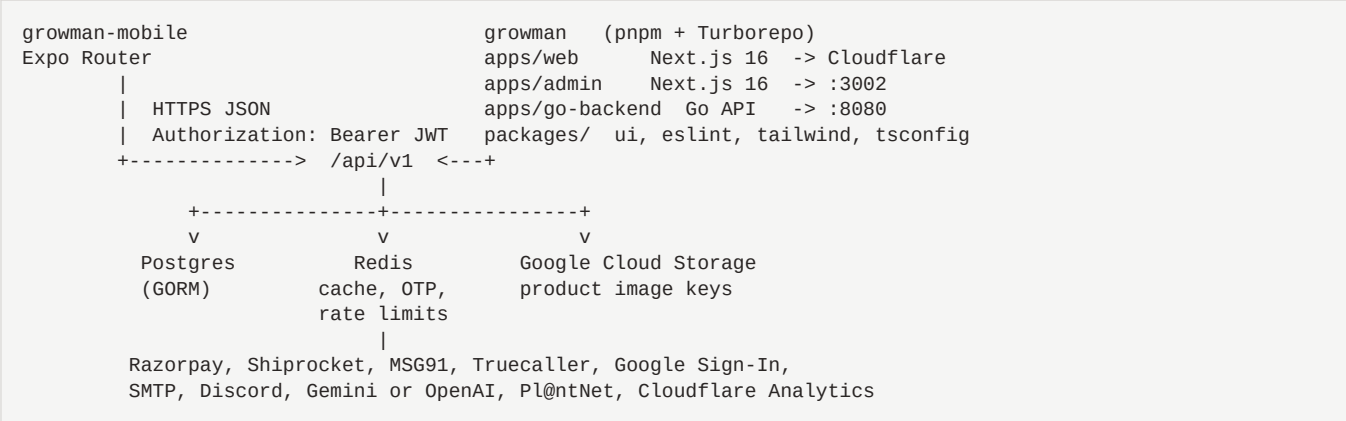


Figure 2. Three clients, one API process, three data stores, and external providers called with plain HTTP.

Runtime ports and production homes

Piece	Local	Production
Website	pnpm --filter web dev, port 3001	Cloudflare Worker growman-web, domain growman.live, via OpenNext
Admin	next dev, port 3002	Separate Next deploy, same API, staff only
API	go run ./cmd/server, port 8080, prefix /api/v1	Cloud Run container
Mobile	npx expo start, then a dev client	Store builds. Bundle id com.venky2100.growman, scheme growman
Postgres and Redis	docker compose at the repo root	Managed Postgres. Redis required for strict limits and OTP in production

Figure 3. Local ports and where each process runs in production.

Why the website and the API are deployed differently

The website is read-mostly HTML, images, and edge caching. Cloudflare Workers plus the OpenNext adapter fit that: catalog pages revalidate, image transforms can be cached at the edge, and a Durable Object queue refreshes the incremental cache. The API is a different shape. It holds a Postgres connection pool, runs three tickers, and must accept webhooks that cannot be frozen between requests. A container on Cloud Run is the simpler place for a process that stays warm. Putting the API on Workers would fight the connection pool and the background workers. Putting the storefront on a long-lived Node server would throw away the edge cache.

Why there is no service layer

One process. One `handlers.Handler` struct holds the database, config, Redis, the image service, and the Shiprocket client. Every endpoint is a method on that struct. Handlers call GORM directly. Reusable pieces are plain functions: query builders, the cache helper, JWT middleware. A repository interface would be mocked in tests that are mostly written as HTTP tests against the handler. The extra files would not have a second caller.

The same reasoning keeps the workers in-process. The inventory sweeper, the shipment worker, and the retention sweeper need the same database and the same clients the handlers already have. A second deployable would be more operational surface than it saves, until a second replica starts booking the same shipment twice. That limit is named in section 18.

Inside one API process

```
HTTPS request
    |
    v
chi router
  RequestID -> RealIP -> SecurityHeaders -> Recoverer
  -> Prometheus -> 60s timeout -> throttle 512 -> gzip
  -> logger -> CORS
  -> per-route: body cap, rate limit, JWT scope or admin role
    |
    v
handlers.Handler method
    |                |                   |
    v                v                   v
cache.Helper      GORM -> Postgres     external HTTP
L1 in-process     users, catalog,      Razorpay, MSG91,
L2 Redis          orders, payments     Shiprocket, Gemini,
singleflight                           Pl@ntNet, Google, SMTP
```

Figure 4. Middleware runs before any handler. Auth is chosen by where the route is mounted, not by a check the handler might forget.

Background workers

Three workers start in `cmd/server/main.go` before the listener, on one cancel context. SIGINT or SIGTERM cancels that context, so shutdown stops the tickers before the connection pool closes. The first pass of each ticker happens one interval after boot, not immediately.

Worker	Interval	Job
StartInventorySweeper	5 minutes	Release stock reserved by orders left unpaid for 30 minutes. Marks the order abandoned. Does not change payment status, so a late capture can still re-reserve.
StartShipmentWorker	15 minutes	Book pending Shiprocket shipments, refresh tracking as a backstop for a missed webhook, ask for a review after delivery.
StartRetentionSweeper	daily	DPDP retention. Drop minimised personal data after the one-year transactional floor.

Figure 5. All three workers share the API process and stop on the same shutdown signal.

5. Technology stack

The split is deliberate. The product UI wants React, because both the website and the mobile app are component trees with local state. The API wants a small static binary, straightforward concurrency, and a single place where stock updates are conditional SQL. Go is used for the API because that is the part that must get money and stock right under concurrent checkouts, with a modest memory footprint. React is used for the screens because Razorpay's browser SDK, Expo native modules, and a shared component style all live in that ecosystem.

Layer	Mobile	Website	Admin	API
Language	TypeScript 5.9, strict	TypeScript 5.9	TypeScript 5.9	Go 1.26
UI	React 19.1, RN 0.81	React 19.2	React 19.2	none
Framework	Expo SDK 54, Router 6	Next.js 16 App Router	Next.js 16 App Router	chi v5
Styling	NativeWind 4, Tailwind 3	Tailwind 4	Tailwind 4	none
State	Zustand 5 + AsyncStorage	Zustand 5 + localStorage	Zustand 5	GORM
Auth token	JWT in Zustand	JWT in Zustand	JWT in Zustand	HS256 JWT, scopes
Payments UI	react-native-razorpay	Razorpay Checkout.js	none	Razorpay REST, no SDK
Deploy	EAS / Gradle bundle	OpenNext to Workers	Next on 3002	Cloud Run
Tests	ESLint	Node tests on lib SEO	ESLint, typecheck	go test ./...

Figure 6. The four runtimes. Admin is a Next app, not a Go UI.

Go dependencies that matter

Module	Why it is here
go-chi/chi/v5 and go-chi/cors	Router and an explicit CORS allow-list
gorm.io/gorm, driver/postgres	ORM. The structs are the schema
go-redis/redis/v8	Cache L2, rate limits, OTP
golang-jwt/jwt/v5	HS256 tokens
golang.org/x/crypto	bcrypt passwords
google.golang.org/api	Verify Google ID tokens
cloud.google.com/go/storage, x/image	Upload to GCS, resize before upload
go-pdf/fpdf	GST invoice and quotation PDFs
prometheus/client_golang	/metrics
golang.org/x/sync	singleflight so a cache stampede is one query

Razorpay, Shiprocket, MSG91, Truecaller, Discord, Pl@ntNet, OpenAI, and Gemini are plain `net/http` clients. They are not SDK dependencies in `go.mod`. That keeps the binary small and makes each integration optional: if the key is missing, that client is nil and only its routes fail.

Monorepo tooling

Web, admin, and the shared packages use pnpm 10 workspaces and Turborepo 2. Shared configs live in `packages/eslint-config`, `packages/tailwind-config`, `packages/typescript-config`, and `packages/ui`. The mobile repository is outside this workspace. It has its own `package.json` and its own lockfile, because Expo and native modules do not belong in the web workspace graph.

There is no shared TypeScript package for API models. `apps/web/lib/types.ts` and `growman-mobile/lib/types.ts` manually mirror the `json` tags in `internal/models/models.go`. Adding a field means updating the struct and both clients. This is a known weakness, stated honestly in section 15.

6. Repositories and code layout

Monorepo

```
growman/
  apps/web/            Next.js storefront, port 3001, worker growman-web
  apps/admin/          Next.js staff dashboard, port 3002
  apps/go-backend/     Go API
    cmd/server/        boot: config, db, redis, workers, listen
    cmd/migrate/       AutoMigrate and exit
    internal/server/   the only route table (router.go)
    internal/handlers/ business logic and the three workers
    internal/models/   every table
    internal/auth/     JWT and middleware
    internal/cache/    L1 + Redis + singleflight
    internal/docs/     system-design essays and the live API reference
  packages/            shared eslint, tailwind, typescript, ui
  docker-compose.yml   Postgres, Redis, Prometheus, Grafana
  docs/                this document and COMPLETE-GUIDE.md
```

API boot order

`cmd/server/main.go` does this in order, and production refuses to start if the money and token secrets are missing:

- **config.Load().** Production will not boot without JWT_SECRET, RAZORPAY_KEY_SECRET, and RAZORPAY_WEBHOOK_SECRET.
- **db.Connect.** DATABASE_URL or HYPERDRIVE_URL. Required. The process does not start without Postgres.
- **db.ConnectRedis.** Nil is allowed. Strict rate limits then fail closed onto an in-process counter. OTP send returns 503.

• **Image service,** only if IMAGE_BASE_URL is set. Implementation is Google Cloud Storage (bucket, project, credentials JSON).

• **handlers.New,** then AutoMigrate when AUTO_MIGRATE is on (the default in development), then search indexes (pg_trgm and partial indexes).

• **Seed** if SEED_ON_STARTUP is set.

• **Start the three workers** on one cancel context.

• **http.Server** with timeouts. Shutdown cancels workers, then drains connections.

Mobile layout

```
growman-mobile/
  app/index.tsx            wait until the auth store has hydrated
  app/_layout.tsx          fonts, gestures, toast, consent banner
  app/(tabs)/              Home, Shop, Cart, Dootha, Account
  app/(auth)/              welcome, phone, OTP, login, signup
  app/<feature>.tsx        stack screens: checkout, product, orders
  lib/                     api.ts plus one module per domain
  store/                   auth, cart, user, search
  components/              screens chrome, product, sheets, skeletons
```

New Architecture is on. Typed routes are enabled. Native modules (Google Sign-In, Razorpay, Truecaller, SMS autofill) mean development uses a dev client, not the Expo Go sandbox. Version in package.json is 2.0.0. Entry is `expo-router/entry`.

7. A request, end to end

A logged-in call from either client follows the same five steps. There is no generated SDK in the middle.

• The screen calls a helper. On both clients that helper is apiFetch in lib/api.ts.

• The helper prefixes the path with the API base, which already includes /api/v1, and sets Authorization: Bearer plus the JWT if a token exists.

• chi runs the global middleware, then the route-group middleware: body limit, rate limit, and the auth middleware that group was mounted with.

• A method on Handler runs. It talks to GORM, the cache, or an external HTTP API.

• JSON comes back. Clients read an error or reason field on failure. readJson on the web is a cast, not a schema validator, which is why field names must match the Go tags.

Two exceptions

Webhooks are not under /api/v1 and they are not authenticated with a user JWT. Razorpay posts to `/webhooks/razorpay` and the body is checked with an HMAC using RAZORPAY_WEBHOOK_SECRET, a different secret from the checkout verify secret. Shiprocket posts to `/webhooks/shiprocket` and is checked with an API key header. Both write a WebhookEvent row. A unique index on the event id makes a replay a no-op that still returns 200, so the provider does not keep retrying a delivery that already succeeded.

Probes

`GET /healthz` is liveness. It checks no dependencies. A liveness probe that touches the database will fail during a database incident, the orchestrator will restart every instance, and a recoverable outage becomes a rolling restart that also throws away the in-process cache. `GET /readyz` is readiness. Postgres down is fatal (503). Redis down is degraded (200), because the process was built to keep serving catalog reads without Redis, and pulling every instance out of rotation would manufacture the outage the design avoids. Checks run under a two-second timeout so a hung database cannot hang the probe. `GET /metrics` is Prometheus. API docs are mounted from the embedded internal/docs tree.

Route groups

The full table is `internal/server/router.go`. The groups, and the middleware each group carries, are the security model. A new route is safe only if it is mounted in the right group.

Group	Middleware	Examples
Probes, metrics, docs, webhooks	No JWT	/healthz, /readyz, /metrics, /webhooks/razorpay, /webhooks/shiprocket
Auth	Strict IP limit	/auth/login, /auth/signup, /auth/google, /auth/otp, /auth/truecaller, /auth/admin/login, forgot-password
Session	Full auth	GET /auth/me, PUT /auth/profile, save location
Onboarding	Onboarding scope only	POST /auth/profile/complete
Catalog	Public, cached	/products, /categories, /brands, /tags, /catalog, /care/season
Checkout	Strict IP limit, optional auth	/checkout/quote, create-order, validate-coupon, email OTP
Payment verify	Strict	POST /razorpay/verify
Guest order	Optional auth plus public token	GET /order, invoice download
Account	Full auth	/orders, /wishlist, /addresses, /privacy, /me/plants, chat history, reorder
Leads	Looser IP limit, fail open	POST /quotations, /dootha-waitlist, /requested-products, /consent
Plant ID	Strict, quota	POST /images/identify-plant and the Pl@ntNet catalog routes
Chat	Strict	POST /chat
Admin	Admin role	Product CRUD, inventory, review moderation, refunds, coupons, shipment, GST report, dashboard map

Figure 7. Auth posture is a property of the route group. Handlers do not each reinvent it.

8. Data model

Every table embeds Base: ID, CreatedAt, UpdatedAt, and DeletedAt. Deletes are soft. `Handler.AutoMigrate` in `internal/handlers/handler.go` is the list of what actually ships. Extra SQL files under migrations/ exist only for indexes and backfills AutoMigrate cannot express, including the trigram search indexes.

Catalog and commerce

Model	What it stores
User	One person. Email and phone unique. Provider is local, google, or phone. Role, location, referral code.
Category, Subcategory, Product	The catalog tree. Product holds price, MRP, stock, TrackInventory, denormalized rating, sizes, attributes.
Brand	Optional on a product.
ProductSize	Per-size price, stock, and image.
Review	Unique per user and product. Status pending or approved. VerifiedPurchase flag.
Order, OrderItem	Razorpay ids, a public token for guests, fulfillment, tax, coupon, AWB, inventory flags.
Payment	One row per Razorpay payment id. Replays upsert instead of inserting a second charge.
Refund, WebhookEvent	Money movement, and the ledger of provider events already handled.
Coupon, CouponRedemption	Server-side discounts: percentage, fixed, or free shipping.
Wishlist, SavedItem	Wishlist is unique per user and product. Saved-for-later is a separate list.
Address	Saved addresses. The product caps a user at ten.

Care, privacy, and leads

Model	What it stores
OwnedPlant, PlantHealthLog	My plants, plus health notes.
NotificationPreference, AppNotification	Inbox and what the user wants to be told.
StoreCredit, CreditEntry	Referral and credit ledger. Not a payment instrument.
SavedPaymentMethod	UPI or card labels the user stored. Not raw card numbers. Razorpay holds the instrument.
RecentView, CartSnapshot, ChatTurn	Engagement and chat memory. CartSnapshot is the last client cart for an abandoned-cart nudge. It is not the cart.
QuoteRequest	Bulk quotation.
RequestedProduct, DoothaWaitlist	Demand: a plant the catalog does not have, and interest in Dootha.
OrderSupportRequest, BackInStockRequest	Support tickets and restock alerts.
ConsentRecord	Append-only DPDP consent and withdrawal.

Figure 8. There is no Cart model. The device is the cart. The server stores orders.

Three patterns that show up everywhere

Snapshot at order time. OrderItem copies the product name and image key. It does not join Product live. A later edit, reprice, or delete does not rewrite a receipt.

Store a key, resolve a URL at the edge. Product images are stored as image_key. Handlers turn that key into a URL with IMAGE_BASE_URL when they write the JSON. The database does not contain a bucket hostname. Moving storage providers does not require a data rewrite.

A unique index is concurrency control. webhook_events.event_id makes webhook dedup atomic without a lock. The review pair (product, user) enforces one review per customer per product. payments.razorpay_payment_id lets a replayed payment upsert. orders.public_token is unguessable and single-purpose, which is what makes a guest order link safe to put in a browser.

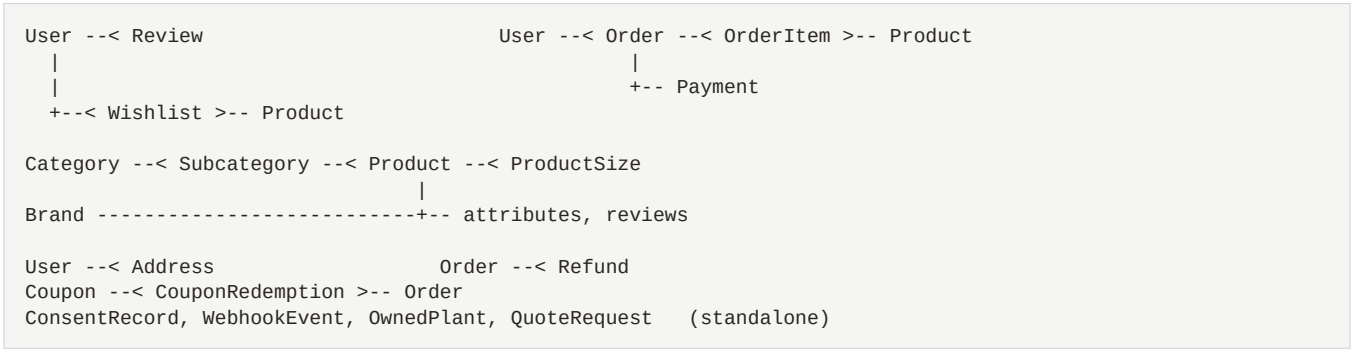


Figure 9. The relationships an interviewer will ask you to sketch. OrderItem is a snapshot, not a live join.

9. Authentication

Four doors, one user. Email and password, Google, phone OTP, and Truecaller all resolve to a row in User and a JWT. The token is HS256, with a 24-hour lifetime. Claims are userId, role, and scope. Passwords are bcrypt. Google ID tokens are verified with the Google API client, not by trusting the client. Phone numbers are normalised in internal/phoneutil before they are stored or compared.

Door	Mobile	Website	API
Email and password	app/(auth)/login.tsx	app/login/page.tsx	POST /auth/login
Google	GoogleLoginButton	GoogleLoginButton	POST /auth/google
Phone OTP	phone.tsx then otp.tsx	Not the primary web path	POST /auth/otp/send and /verify
Truecaller	lib/truecaller.ts, Android	Not offered	POST /auth/truecaller
Finish a new phone profile	complete-profile.tsx	Not offered	POST /auth/profile/complete
Staff	Not offered	Admin /login, Turnstile	POST /auth/admin/login

Figure 10. Every successful door returns the same token shape.

Scope and role

scope=full is a normal session. FullAuthMiddleware wraps account routes. **scope=onboarding** is a phone or Truecaller user who does not yet have a name and email. OnboardingAuthMiddleware only lets that token hit profile completion. The user cannot place an order and cannot read /auth/me as a full user. Scope is enforced where the route is mounted. A check that lived inside each handler would only protect the handlers that remembered it.

role is customer, admin, or superadmin. AdminMiddleware allows admin and superadmin. There is no second staff directory. A staff login that does not carry one of those roles is rejected even if the password is correct. The admin UI also wraps pages in AdminAuthGate, which is a client-side redirect. The API is what actually rejects a missing or wrong token.

Where the token lives

Both customer clients store the JWT in Zustand, persisted under the key auth-storage. On the web that is localStorage. On mobile it is AsyncStorage. It is not an httpOnly cookie. That is a parity choice with mobile, which has no cookie jar shared with the API. The trade-off is real: an XSS bug on the website can read the token. Route protection in the browser is a convenience. The API re-checks the token on every call.

On mobile, setToken starts userStore.loadUser(). resolveAuthToken reads Zustand first and then AsyncStorage, so a request fired before hydration can still find the token. The website does the equivalent with its auth store. Guest checkout does not need a user id. It needs a single-use email proof minted by verify-email-otp and bound to that address.

SAY THIS IF THEY ASK ABOUT SESSIONS

We do not have server sessions. We have a signed JWT with a scope, checked on every request, plus two narrow capability tokens: the guest order public token, and the checkout email-verification proof. Neither capability token is a session. Neither widens because the holder is logged in.

10. Catalog, search, cache, and images

Reads dominate, so the catalog is cached aggressively and the product page is the exception that opts out where stock must be live. A cached product read goes router, then GetProduct, then cache.GetOrLoadRaw. That helper checks an in-process map, then Redis. On a full miss, singleflight makes sure only one caller queries Postgres. The loader fans sizes, attributes, reviews, category, subcategory, and brand out with an errgroup instead of five sequential preloads, because each round trip to a remote database costs real time. The result is written back through both tiers and served with an ETag, so an unchanged client gets 304.

Two tiers

Tier	Where	Behaviour
L1	In-process map, about 2 minutes, 4096 keys	Hot keys avoid a Redis round trip. Each API replica has its own L1. That is acceptable because Redis is the shared tier.
L2	Redis	Shared across replicas. Optional. A nil client means L1 only, which is slower and still correct for reads.
Coalescing	singleflight	N concurrent misses on one key become one database query.
HTTP	ETag on public catalog responses	Repeat views of an unchanged payload skip the body.

TTLs follow how fast the data changes. A single product detail can sit longer than a search result. A user's order list is short, because status changes are the reason the customer is looking. Account payloads use a private cache helper or no cache. Admin writes must invalidate catalog keys. A write that forgets invalidation is a bug that looks like a successful save and a stale shop.

Search

Free-text search uses Postgres with pg_trgm indexes created by EnsureSearchIndexes after migrate. Filters cover category, subcategory, brand, tag, and price-shaped attributes exposed by the filters endpoint. The website shop page is a server-rendered listing. Mobile keeps a 10-minute in-memory catalog cache in lib/catalog.ts, because it has no edge cache and no server components.

Images

Upload is an admin route. The handler resizes with golang.org/x/image and writes the bytes to GCS through internal/services/storage. The database stores the key. Every read path resolves ImageURL from IMAGE_BASE_URL. If storage is down, uploads fail and already-stored keys resolve to an empty URL instead of taking the API down. The shop still works without pictures.

What the website adds on top

Say the website caches in this order if they ask why the homepage is fast:

- Next.js revalidate, or no-store, on the fetch. Catalog listings use about 10 minutes. The product page uses no-store so price and stock are not served stale, and React cache() so metadata and the body share one fresh request.
- OpenNext incremental cache in Cloudflare KV, plus a regional Cache API.
- A Durable Object queue that revalidates on a timer.
- Cloudflare Images and a worker cache for /_next/image.
- The service worker and the static asset bucket for the PWA shell.

11. Checkout, stock, payments, tax, and shipping

This is the part of the system to draw on a whiteboard. The client sends what it wants to buy. The server decides what it costs and whether the stock exists. Coupon math, GST, delivery fee, and the Razorpay amount are computed in the API. Signature checks use two secrets, one for the browser callback and one for the webhook, and production will not boot if either secret is missing.

```
Cart on the device  (Zustand, not a table)
    |  POST /checkout/quote             fee, zone, pickup vs courier
    |  POST /checkout/validate-coupon   server recomputes the discount
    |  guest: send-email-otp + verify-email-otp
    v
POST /checkout/create-order
    |  re-price every line from the product and size rows
    |  reserve stock inside the same transaction
    |  create Order, OrderItems, Razorpay order
    v
Razorpay checkout  (Checkout.js on web, react-native-razorpay on mobile)
    |
    +-- POST /razorpay/verify          client callback, HMAC, fails closed
    +-- POST /webhooks/razorpay        provider callback, other secret, idempotent
              |
              v
        captured  -> stock committed, confirmation email
        unpaid 30 min -> sweeper releases the reservation
        paid -> shipment worker books Shiprocket, AWB, tracking webhook
        invoice -> GST PDF, Indian financial year April-March
```

Figure 11. Checkout. The two arrows into mark-paid can race. Only the first one sends mail and commits stock.

What create-order refuses to trust

create-order validates the shipping address, resolves every line against the database in batched queries, and takes each line's price from the product or size row. The price in the JSON body is ignored. A guest must present the single-use proof token that verify-email-otp minted for that exact address. The fulfillment channel is required, because the server re-runs the delivery quote against it. Omitting the channel used to be a way to skip repricing, so the field is now mandatory. Any coupon is evaluated again on the server. Only then does the handler create the Razorpay order. The local Order, its items, the stock reservation, and the coupon redemption commit in one database transaction. An order without a reservation would oversell. A reservation without an order would leak stock.

The database arbitrates

Two shoppers can reach the last plant at the same moment. Two checkouts can try to use the last redemption of a capped coupon. The browser callback and the webhook can both report the same payment. The application does not take a mutex and it does not take a Redis lock. A Redis lock can fail open, which is the wrong direction for an oversell or a double refund. Postgres is already a single primary that serialises these statements. The check lives inside the statement that acts, so there is no window between deciding and doing.

Transition	Statement	If you lose
Order becomes paid	UPDATE orders SET payment_status='paid' WHERE id=? AND payment_status <> 'paid'	RowsAffected is 0. Skip email and stock commit. The mail goes out once.
Reserve stock	UPDATE products SET stock = stock - ? WHERE id=? AND (track is off OR stock >= ?)	RowsAffected is 0. Roll the order back. Respond 409 out_of_stock.
Release stock	UPDATE orders SET inventory_released=true WHERE id=? AND inventory_released=false	The release is claimed before units are credited, so a replay cannot inflate stock.
Webhook dedup	INSERT into webhook_events on a unique event id	Unique violation. Return 200 with duplicate true. Do not reprocess.
Coupon cap	UPDATE coupons SET usage_count = usage_count + 1 WHERE usage_count < limit	RowsAffected is 0. Reject. Two checkouts cannot push a global cap past its limit.

Figure 12. Each race is a conditional write. RowsAffected tells the handler whether it won.

Reserve, commit, release

Stock is held from order creation, not from payment. The window between the two is minutes, and a customer who has reached the payment screen should not lose the plant to someone who is still browsing. The cost is that an abandoned checkout holds stock it may never buy. The sweeper exists to undo that.

Reserve happens inside the order transaction. Commit happens on the paid transition. Usually the units are already deducted and commit only stops the sweep from clawing them back. The awkward case is a payment that lands after the sweep already returned the stock: a slow bank redirect, a tab left open. Clearing the flag would mark an order paid against units that are back on the shelf. So a released reservation is taken again. If the units are gone, the order still stands, because the money has been taken, and a support request is raised for a person to source the plant or refund. An oversell someone is told about is better than one discovered at packing time. Release happens on payment failure, cancellation, or the sweep after 30 minutes unpaid. The sweep marks the order abandoned and leaves payment_status alone, so markOrderPaid can still re-reserve.

Fulfillment and GST

internal/fulfillment is the authority on zones, channels, and fees. The quote uses the nursery coordinates and pincode (STORE_LATITUDE, STORE_LONGITUDE, STORE_PINCODE) and the customer's address. Channels include courier and store pickup. The fee thresholds live in configuration, not in the client. After payment, the shipment worker calls Shiprocket: serviceability, order create, AWB, pickup. Tracking updates arrive on the Shiprocket webhook, and the worker refreshes tracking on its own interval in case a webhook is missed.

The invoice is a PDF rendered with go-pdf/fpdf. It is a GST tax invoice, numbered per Indian financial year (April through March), not the calendar year. Amounts are GST-inclusive. A monthly GSTR-ready CSV is an admin export at GET /reports/gst. The customer downloads the PDF from the order, on the web and on mobile (mobile uses the file system and the share sheet). Staff can also email the invoice.

Files to name

checkout.go, payments.go, webhooks.go, inventory.go, coupons.go, fulfillment.go, invoicing.go, shipments.go, internal/shiprocket, internal/fulfillment, and services/invoice_pdf.go. On the clients: app/checkout on both, lib/fulfillment.ts, cartStore, and the Razorpay button component for that platform.

12. The website

apps/web is the customer storefront. Package name web. Next.js App Router only. There is no pages/ directory. It is a mix of React Server Components for anything public and indexable, and client components for anything that needs the token, the cart, or a click handler.

How a page gets data

Kind of page	Pattern	Examples
SEO catalog	Async Server Component. Fetch revalidates about 10 minutes.	Home, shop, category listings, sitemap
Product detail	Server Component plus client islands. Product fetch is cache() and no-store.	app/product/[slug]/page.tsx
Account, cart, checkout	use client. Bearer token from Zustand.	Cart, checkout, orders, account

lib/api.ts chooses a base URL. In the browser that is NEXT_PUBLIC_GO_API_URL, then NEXT_PUBLIC_API_URL, then localhost:8080/api/v1. On the server it prefers GO_API_URL, which is not exposed to the browser. The Razorpay secret never appears in this app. Only the key id is public.

Routes a customer actually uses

URL	Role
/	Hero, featured plants, categories. Revalidate 10 minutes.
/shop, /categories/...	Filters, pagination, category and subcategory listings. Category level can be statically generated.
/product/[slug]	Product page: JSON-LD, gallery, size selector, reviews, add to cart.
/search	Client search, plus a way to request a plant the catalog does not have.
/cart, /checkout, /order-success	Local cart, quote, coupon, guest OTP, Razorpay, confirmation.
/orders, /wishlist, /account	History, cancel, invoice, wishlist, profile, addresses (max 10), privacy export and erase.
/login, /signup, /forgot-password	Email, password, Google. Turnstile on sensitive forms.
/bulk, /gifting	Quotation builder (POST /quotations) and gifting collections.
/blog, /about, /help-center, /privacy-policy, /terms	Content. Blog posts are static data in lib/seo/blog.ts.
sitemap.ts, robots.ts	Public URLs are listed. Cart, checkout, account, and login are disallowed.

Figure 13. Public pages are server-rendered for search engines. Private pages are client-rendered and noindexed.

SEO and the edge

Metadata is built in lib/seo. Product pages emit JSON-LD so a search result can show price and availability. The sitemap is generated from static paths plus the catalog. robots.txt keeps transactional URLs out of the index. Security headers (HSTS, nosniff, frame ancestors, COOP, and a tight camera, microphone, and geolocation policy) are set in next.config.ts.

Deployment is wrangler.jsonc, worker name growman-web, entry worker.ts, custom domain growman.live. Bindings include a rate-limit KV, the OpenNext incremental-cache KV, Analytics Engine, Cloudflare Images, and a Durable Object for cache revalidation. worker.ts wraps the OpenNext worker and edge-caches image transforms. middleware.ts is not an auth gate. On the Cloudflare runtime it rate-limits with KV, drops obvious bots, and writes analytics cookies. In next dev those bindings are absent and that work does nothing. Authorization remains the API's job.

UI building blocks

Home lives in components/hompage (hero, navbar, plant sections). Catalog UI lives in components/productspage. The product page is split into a server page and client islands: add to cart, size selector, tabs, reviews. Chat is components/chatbot/PlantChatbot.tsx. Privacy is a consent banner plus account export and erase. Pickup is a store dialog fed by the same quote endpoint the mobile app uses. The PWA shell is public/sw.js and public/manifest.json, registered in production. Toasts go through sonner, wrapped by lib/toast.tsx. Motion uses framer-motion.

13. The mobile application

growman-mobile is the same shop, built for a phone, with the Indian identity methods and the care reminders that a browser handles poorly. It speaks the same /api/v1 and the same JWT. It does not server-render. Navigation is file-based Expo Router. Styling is NativeWind, which is Tailwind 3 class names on React Native views. The tab bar green is #059669. Display type is Fraunces and body type is Manrope, loaded before the splash screen hides.

Navigation

app/index.tsx waits until the auth store has hydrated, then redirects to the home tab or to the auth stack. The root layout wraps the tree in a gesture handler, loads fonts, and mounts toast, alert, and the consent banner. The tab bar is Home, Shop, Cart, Dootha, and Account. The Dootha tab does not push a screen. Its tab press calls openChatbot(), and components/Chatbot.tsx draws a sheet. app/(tabs)/chat.tsx is only a placeholder. Feature screens (checkout, product, orders, my plants) are stack routes on top of the tabs.

Screens

Route	What the user does
(tabs)/home, shop, cart, account	Featured plants and scan entry; browse; local cart; profile hub and logout.
(auth) welcome, phone, otp, login, signup	Google, Truecaller, phone OTP, email password. Signup draft is held in signupDraftContext.
complete-profile	Name and email after a new phone user. Token scope is onboarding until this succeeds.
product/[id], category, categories	Product detail accepts an id or a slug. Listings by category.
checkout, order-success, orders	Quote, pay with Razorpay, confirmation, history, invoice share, retry.
wishlist, saved, saved-addresses, payment-methods	Wishlist versus save-for-later, addresses, stored payment labels.
my-plants, calendar, edit	Owned plants, water and fertilize, health, care calendar.
care-guides, plant-finder	Static articles from lib/careGuides.ts. Filters for light, pets, and climate.
request-plant, bulk, gifting	Demand and wholesale.
your-data, delete-account, notifications	DPDP export and erase. In-app inbox.
about, support, help-center, terms, privacy	Content and legal pages.

Figure 14. Mobile routes. Care and phone identity are first-class here.

State

Store	Persisted	Holds
authStore	Token only, key auth-storage	token, isAuthenticated, hasHydrated
cartStore	The whole cart, key cart-storage	Line items and quantity helpers
userStore	localLocation only	Profile loaded from GET /auth/me
searchStore	No	Whether the search modal is open, and a pending query

Zustand is used when two screens share the data or it must survive a restart. Otherwise a screen uses useState and refetches on focus. Domain helpers live in lib, not inside the screen: catalog.ts (10-minute memory cache), fulfillment.ts, plantCare.ts, careReminders.ts, chatApi.ts, quotation.ts, consent.ts, downloads.ts. Cart sync via POST /me/cart-signals is analytics. The cart on disk is the cart.

What mobile has that the website does not

- Phone OTP with MSG91, Android SMS autofill, and the phone-number hint.
- Truecaller one-tap when TRUECALLER_ANDROID_CLIENT_ID is set.
- A camera flow for Pl@ntNet and for attaching a photo in chat or search.
- Local notifications on a channel named care. Tapping a reminder opens /my-plants/:id. Scheduling is on the device in careReminders.ts. The server still owns the plant record.
- Voice search through expo-speech and speech recognition.
- Invoice and data-export files shared through the system sheet.
- Location permission used when the customer locates an address.

Configuration is EXPO_PUBLIC_* in config/env.ts: API URL, Google client ids, Razorpay key id, MSG91 widget ids, Truecaller client id. A physical device cannot use localhost. EXPO_PUBLIC_API_URL must be the machine's LAN address or the Cloud Run URL. app.config.js injects the Truecaller client id and release signing from the environment. Plugins cover the router, splash, location, Google Sign-In, speech recognition, Truecaller, and notifications.

MOBILE VERSUS WEBSITE, IN ONE BREATH

Same API, same JWT, same rule that the server prices the cart. Mobile adds phone OTP, Truecaller, SMS autofill, a local care-notification schedule, and a camera for plant identification. Navigation is Expo Router instead of the App Router. Styling is NativeWind on Tailwind 3 instead of Tailwind 4. There is no server render and no edge cache. The catalog cache is an in-memory map for ten minutes.

14. The admin dashboard

apps/admin is a Next.js 16 application on port 3002. It uses the same monorepo packages, Zustand, and Tailwind 4. It is not a separate backend and it is not a customer surface. Staff sign in through POST /auth/admin/login. Cloudflare Turnstile sits on that login. The client gate is AdminAuthGate. The real gate is AdminMiddleware on the API: the JWT role must be admin or superadmin.

Screens

Route	What staff do
/login, /forgot-password	Staff sign-in with Turnstile, and password recovery. The navbar hides itself on these routes.
/	Dashboard. Metric cards for products, categories, brands, and tags. A filterable orders table. A choropleth of request volume.
/products	Catalog list and search.
/product/add	Create a product, including image upload through the admin image route.

The dashboard home is a client page. It loads product, category, brand, and tag counts, then a paginated order list (20 per page) with status filter, order-id search, and customer search. Staff can update order status and expected delivery date from the table. Status values include pending, confirmed, shipped, out for delivery, delivered, cancelled, failed, and paid.

The request map

TrafficChoroplethMap.tsx draws demand with d3 and topojson. The page calls GET /dashboard/map with a map type (world or country), a country, and a time frame. The handler in dashboard.go reads Cloudflare Analytics using CLOUDFLARE_API_TOKEN, CLOUDFLARE_ZONE_ID, and CLOUDFLARE_ACCOUNT_ID. The map is an operational picture of where requests come from. It is not a second analytics product, and it does not replace order data in Postgres.

What the admin API can do beyond the current screens

The UI concentrates on the dashboard, the order table, and catalog editing. The admin route group on the API is wider, and a viva answer should separate the two. These endpoints exist and are role-gated even where the dashboard does not yet have a dedicated page:

- Product create, update, and delete, plus image upload.
- Inventory adjust and a low-stock list.
- Review moderation: list pending reviews and patch status. Approved reviews roll a denormalized rating back onto the product. Verified-purchase is computed, not sent by the client.
- Refunds: list them, and create one against an order.
- Coupons: list, create, update, delete.
- Shipments: create a Shiprocket shipment and schedule pickup.
- GST report export, and emailing an invoice.
- Quote status updates for bulk quotation requests.

The navbar also searches products and deep-links the public product page on growman.live, so staff can see the customer view of a slug without leaving the dashboard. robots.ts keeps the admin host out of search indexes.

WHERE A NEW STAFF FEATURE GOES

If only staff should see it, the screen goes in apps/admin, the route goes in the admin group in router.go, and the customer apps do not get a button. Do not hide an admin action behind a client-side check on the public website.

15. Dootha, plant care, and photo identification

Dootha

Dootha is the plant-care assistant. POST /chat classifies intent, answers care questions, and can attach products from the catalog. When the JWT is present it may read that user's orders and wishlist. It cannot place an order, change a price, or issue a refund. History for a logged-in user is GET /me/chat, stored as ChatTurn rows. The provider is selected with AI_PROVIDER. Gemini is the default because the free tier keeps the cost of a care question low. OpenAI is the alternative. If the provider is down or the key is missing, the handler returns a canned keyword answer from getFallbackResponse. Chat never 500s into a blank bubble.

On mobile, Dootha is a bottom sheet opened from the tab bar. On the website it is PlantChatbot. Both call the same endpoint. The route sits behind a fail-closed rate limit because each turn can spend a paid model call.

My plants

Owned plants are account data under /me/plants: create, read, update, delete, water, fertilize, health log, care calendar, and import from past orders. Season copy comes from internal/care and is not a database table. It is advice keyed by season, shared by both clients through GET /care/season. Mobile schedules local notifications from that data. The server remains the record of the plant. Uninstalling the app does not delete the plant. It does drop the local reminder until the app is opened again and the schedule is rebuilt.

Photo identification

POST /images/identify-plant sends a photo to Pl@ntNet. Related routes expose species, diseases, varieties, languages, projects, and the remaining quota. internal/plantnet holds the client and a daily quota policy, with PLANTNET_RESERVE keeping a buffer. The route is fail-closed and quota-limited. A missing key or an exhausted quota is a clear error, not a silent call. Mobile uses expo-image-picker. The website can offer the same endpoint where a file input exists. Identification is a suggestion. It does not create an order and it does not write a product.

Care guides and the plant finder

Care guide articles on mobile are static content in lib/careGuides.ts. The plant finder filters by light, pets, and climate against the catalog. These are read features. They do not need a new table. The website's blog is the same idea: static posts, metadata, and a sitemap entry.

16. Privacy, security, and failure

DPDP

The Digital Personal Data Protection Act 2023, and the rules that followed, ask for consent, access, and erasure. They also expect transactional records to be kept long enough to defend a payment. Those two requirements conflict if erasure means deleting the order row today. Growman records consent in ConsentRecord (POST /consent). Export and erasure are authenticated routes under /privacy. Erasure removes or minimises personal data that is not inside the retention floor. A daily sweeper is what actually ages minimised order data out after one year. The order as an accounting document is not thrown away on the day the customer taps delete.

The website surfaces this in the account privacy page and a consent banner. Mobile surfaces it in your-data and delete-account. Both call the same API. The client does not delete rows in Postgres by itself.

Security model

- Authentication is the JWT described in section 9. Authorization is the role check, applied as middleware.
- Guest order access uses Order.PublicToken, an unguessable handle for one order, not a session.
- Checkout email verification is a single-use proof bound to one address.
- Webhook authenticity is HMAC or an API key, never a user token. The two Razorpay secrets are different.
- Security headers middleware runs on every API response. The website adds HSTS and a frame policy at the edge.
- CORS is an allow-list from CORS_ORIGINS, not a reflection of any origin.
- Body size on /api/v1 is capped (about 11 MB) so an upload cannot pin the process. A global throttle caps in-flight requests.
- Clients cannot set their own role, user id, or price in a body and have it honoured.
- Saved payment methods store labels. Card numbers stay at Razorpay.

Rate limits, fail open and fail closed

A global IP limit on /api/v1 is about 250 requests a minute. Auth, checkout, plant identification, and chat add stricter limits. Catalogue reads fail open: if Redis is down, the worst case is extra load on Postgres, and refusing to show the shop is worse than serving it. Login, OTP, checkout, chat, and plant identification fail closed. They guard credentials, reserve inventory, or call a paid provider. An outage that makes credential stuffing free, or an AI bill unbounded, is not an acceptable degradation.

Fail closed does not mean reject everything, because that would hand an attacker a cheap outage. The bucket falls back to an in-process fixed-window counter. With N replicas the effective limit is about N times the configured one, and the windows are not aligned. N times a budget is still a budget. Failing open is not. Quotation and waitlist limits fail open on a Redis blip, because losing a sales lead is worse than accepting one extra form post.

When a dependency is down

Dependency	What the customer sees
Redis	Catalog still serves from L1 and Postgres. OTP send returns 503. Strict limiters use the local counter. readyz reports degraded.
GCS	Uploads fail. Existing pages load with empty image URLs. The API stays up.
SMTP	OTP and password reset return 503 before they pretend to send. Order mail is fire-and-forget, so a payment still succeeds if the confirmation email fails.
Gemini or OpenAI	Dootha answers from the keyword fallback. No blank error.
Razorpay	Checkout fails loudly. There is no safe fake payment.
MSG91, Truecaller, Google, Pl@ntNet	Only the route that needs that provider fails. Nothing else is constructed unless it is configured.
Postgres	The process will not boot, and readyz is 503. There is no catalog without it.

Figure 15. Optional dependencies are nil until configured. A missing key degrades one feature.

17. Deployment and local setup

Production

The website builds with OpenNext and deploys with Wrangler to the worker growman-web. The API is a container: the Dockerfile in apps/go-backend produces the server binary, and Cloud Run runs it. Admin is a separate Next.js deploy pointed at the same API base URL. Mobile release builds are produced with EAS or with the Gradle bundleRelease script in the mobile package. Images live in a GCS bucket. Postgres is managed. Redis is required in production for OTP and for the strict limiters to be shared across replicas. Prometheus metrics are scraped from /metrics. The compose file also defines Grafana for local dashboards.

Environment names, not values

Secrets are not written in this document. The names an interviewer may ask for are: GO_PORT, GO_ENV or APP_ENV, AUTO_MIGRATE, DATABASE_URL, REDIS_URL, JWT_SECRET, CORS_ORIGINS, GOOGLE_CLIENT_ID, RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET, SMTP settings, IMAGE_BASE_URL and the GCS trio, OPENAI_API_KEY, GEMINI_API_KEY, AI_PROVIDER, PLANTNET_API_KEY, MSG91 and TRUECALLER keys, SHIPROCKET keys, store coordinates, and the Cloudflare analytics token for the admin map. Templates live in apps/go-backend/example.env and apps/web/example.env. Browser-visible website values must be prefixed NEXT_PUBLIC_. Mobile values must be prefixed EXPO_PUBLIC_.

Running it locally

```
cd /home/venky/growman
docker compose up -d                 # Postgres, Redis, Prometheus, Grafana

cd apps/go-backend
cp example.env .env                   # set DATABASE_URL and JWT_SECRET
go run ./cmd/server                   # :8080   AUTO_MIGRATE on in dev

cd /home/venky/growman
pnpm install
pnpm --filter web dev                 # http://localhost:3001
pnpm --filter admin dev               # http://localhost:3002

cd /home/venky/growman-mobile
npm install
# EXPO_PUBLIC_API_URL=http://<lan-ip>:8080/api/v1
npx expo start
```

Health check is curl http://localhost:8080/healthz. Dev payment verification can be skipped only when APP_ENV is development. Do not point a release build at a development database. A physical phone must use the LAN address, not localhost.

18. Testing, limits, and what changes at ten times the traffic

What is tested

The API is where the tests that protect money live. go test ./... covers payments, inventory, invoicing, fulfillment, plant image handling, JWT, rate limits, security headers, cache, care season, the Pl@ntNet client, the Shiprocket client, Discord notifications, the OTP proof, quotations, and the docs handler. Most are table-driven httptest cases. A new behaviour that touches stock, money, auth scope, or a webhook needs a test in that style.

The website runs a Node test runner over lib SEO helpers (metadata, sitemap rules, JSON-LD). Admin and mobile are linted and, for admin, typechecked. Mobile does not have a Jest suite. That is a gap, not a hidden test folder.

Honest weaknesses

- Web and mobile duplicate types. Nothing checks them against the Go struct at build time. A renamed JSON field fails at runtime.
- Auth tokens are readable by JavaScript. An XSS bug on the website can steal a session. The API still re-checks every call, which limits the blast radius to that token's lifetime and scope.
- Mobile has no automated tests. Regressions in checkout on a phone are caught by using the app.
- AutoMigrate on boot is convenient and dangerous if a bad struct ships. A dedicated migration tool becomes the right step once a production change can lock a hot table.
- The Handler struct keeps growing. That is acceptable while there is one process. It becomes the wrong shape when a second deployable needs the same stock rules.
- L1 cache is per process. That is fine. N replicas each running the shipment worker is not fine, because they can race to book the same shipment. The unique constraints catch some of this. A single leader should own the tickers before the replica count grows.

At ten times the traffic

API replicas are already stateless behind a load balancer, apart from the in-process cache and the in-process rate-limit fallback. The next pressure points, in order, are a Postgres read replica for catalog queries, moving the three sweepers to a single leader so N replicas do not all book the same shipment or sweep the same orders, and replacing boot-time AutoMigrate with versioned migrations. Microservices are not on that list. The stock rules need one writer more than they need a network boundary.

19. How a feature is added

The order is always the API first when the feature persists anything or changes money. Doing it only on the client is how price and privacy bugs land.

Kind of feature	Server	Client
Static page, guide, FAQ	None	A route on web and or mobile. Sitemap and metadata on the web.
Read of existing data	Maybe a query parameter	A helper and a screen.
New account data	Model, AutoMigrate, handler, route, auth middleware	Both clients if both should show it.
Money, stock, coupons, invoices	Extend checkout, inventory, or payments. Add a Go test.	Display the server response. Do not recompute it.
Paid third party	Strict rate limit, quota, fail closed	A clear error when the quota or the key is missing.
Staff only	Admin middleware	apps/admin only.

API checklist

- Add the struct to internal/models/models.go with json and gorm tags. Embed Base.
- Add the type to AutoMigrate. Add a SQL file only when AutoMigrate cannot express the index.
- Write methods on Handler. Take the user from the auth helper. Respond with pkg/httpjson.
- Register the route in router.go inside the right group. This is the step that picks the security posture, and the step people forget.
- New environment variables go on the config struct, in Load, and in example.env. Document the name. Never commit a value.
- Public catalog payloads are cached and invalidated on the admin write. User data is not put in the public cache.
- go test ./... and a focused test if the feature touches stock, money, scope, or a webhook.

Client checklist

On the web, a public indexable page is a Server Component with metadata and a sitemap entry. A private page is a client component, noindexed, and it sends logged-out users to /login with a redirect. On mobile, the screen is a file under app/, navigation is router.push, the helper is lib/feature.ts, and unauthenticated users who would get a 401 are sent to the auth stack. Both clients send the same JSON field names. Both show loading, empty, and error states.

Questions before a change is finished

- Who can call this if they are logged out, onboarding-only, or a different user?
- Can they set their own price, role, or user id in the body?
- What happens when Redis is nil?
- What happens when the external provider times out?
- Did the catalog cache get invalidated, or did a private payload get cached?
- Do web and mobile send the same JSON?

20. Interview answers

Say these in your own words. The paragraph under each question is what you expand if they push. Read this section the evening before a viva. Sections 11 and 16 are the ones to be able to draw.

What is the system?

Three clients, one Go API. The website is Next.js on Cloudflare. Mobile is Expo. Admin is a second Next app for staff. Postgres is the source of truth. Redis is cache, OTP, and rate limits. Images are keys in Google Cloud Storage. Payments are Razorpay. Shipping is Shiprocket. The public site is growman.live.

Why Go for the API and React for the screens?

The API is the part that must get stock and money right under concurrency, as one binary with a small memory footprint. The screens want React: Razorpay's browser SDK, Expo native modules, and a component UI. Splitting them lets each side use the ecosystem it needs. The contract between them is JSON under /api/v1.

Why is there no service or repository layer?

One process, one caller. Handlers are the service. A repository interface would be mocked in tests we mostly write as HTTP tests. The moment a second deployable needs the same stock rules, those rules should move out of the handler. That moment has not arrived.

How does authentication work?

Email, Google, phone OTP, and Truecaller all become one User. The API returns an HS256 JWT with userId, role, and scope. Full versus onboarding stops a half-registered phone user from hitting account routes. Admin is a role check, not a second user table. Clients keep the token in Zustand. The API re-checks it on every call. Website middleware does analytics and edge rate limits, not authorization.

How do you stop two people buying the last plant?

create-order reserves stock with a conditional UPDATE inside the order transaction, before Razorpay opens. Capture commits the reservation. Cancel, failure, or the five-minute sweeper after thirty minutes unpaid releases it. If payment arrives after the sweep, the handler tries to take the stock again and opens a support request if it cannot. A unique key on webhook events stops a retried Razorpay event from committing twice. The same conditional-update idea caps coupons and refunds.

Why can the app not send the price?

The device cart is a convenience cache. It can be stale or edited. create-order loads the product and size rows and computes subtotal, coupon, delivery, and GST. Razorpay is opened for that computed amount. Verify and the webhook both check signatures, with different secrets. Production refuses to boot if either secret is missing.

How is the catalog fast?

Reads dominate. Two-tier cache plus singleflight, then ETags. The website also revalidates HTML for about ten minutes at the edge, but the product page opts out so stock is live. Admin writes must invalidate the keys. Mobile has a ten-minute in-memory cache because it has no edge.

What if Redis is down?

Catalog cache misses fall through to the in-process cache and then Postgres. OTP and the fail-closed limiters do not silently allow password guessing or unpaid PlantNet calls. They use a local counter or they reject. Lead forms fail open. The process stays up. readyz goes degraded so a load balancer can choose to keep sending traffic, which is what we want, because the shop still works.

Why Cloudflare for the site and Cloud Run for the API?

The site is read-mostly HTML and images. Workers and OpenNext fit that, including a KV incremental cache and a Durable Object revalidation queue. The API has a long-lived Postgres pool, background tickers, and webhooks that must not be frozen between requests. A container is the simpler place for that.

How does mobile differ from the website?

Same API and same JWT. Mobile adds phone OTP, Truecaller, SMS autofill, local care reminders, and a camera for Pl@ntNet. Navigation is Expo Router. Styling is NativeWind. There is no server-side render. The cart and the token survive in AsyncStorage.

What is Dootha?

The chat assistant. POST /chat answers care questions, can attach products, and, when a JWT is present, can read that user's orders and wishlist. The provider is Gemini by default, OpenAI if configured, and a keyword fallback if the provider fails. It is a sheet on mobile and PlantChatbot on the web. It does not place orders.

How do you handle Indian specifics?

Razorpay rather than a card-network checkout aimed at the US. MSG91 for SMS. Phone numbers normalised before storage. GST-inclusive invoices numbered on the April to March year, plus a GSTR-ready CSV. Shiprocket for pincodes, AWB, and pickup. Delivery zones from the nursery's own coordinates. Rupee formatting on the clients.

What about the DPDP Act?

Consent is stored. Export and erasure are authenticated. Erasure does not throw away the accounting record inside the one-year floor. A daily sweeper minimises personal data after that floor. Delete account on the phone calls the API. It does not drop tables from the client.

What is the admin dashboard?

A Next.js app on port 3002 for staff. Login is a separate admin endpoint plus Turnstile, and the API checks the admin role. The home page shows catalog counts, an orders table staff can update, and a choropleth of request geography from Cloudflare Analytics. Catalog create and edit live here. Heavier operations (refunds, coupons, GST export, shipment booking, review moderation) are admin routes on the API.

How would you add a feature?

Decide whether it is content, a read, new account data, money, a paid third party, or staff-only. Then model if needed, a handler method, one line in router.go with the right middleware, then a thin client helper and a screen on each app that should show it. Money features get a Go test and do not trust the client total.

What would you change first if traffic grew?

A read replica for catalog, a single leader for the three workers, and versioned migrations instead of AutoMigrate on boot. Not a microservice split.

What is weak today?

Duplicated types with no schema check. Tokens readable by JavaScript. No mobile test suite. AutoMigrate on boot. A Handler that will keep growing until a second service needs the same rules. I would say these before an interviewer finds them.

21. File index

When an interviewer asks where something lives, these are the first files to name. Paths are relative to the growman monorepo unless they say mobile.

Question	First file
Where is a route mounted?	apps/go-backend/internal/server/router.go
How does the process boot?	apps/go-backend/cmd/server/main.go
What are the tables?	apps/go-backend/internal/models/models.go
How is the JWT built?	apps/go-backend/internal/auth/jwt.go
Why is the system shaped this way?	apps/go-backend/internal/docs/SYSTEM-DESIGN.md
Checkout and stock	internal/handlers/checkout.go and inventory.go
Razorpay verify and webhooks	payments.go and webhooks.go
GST PDF	invoicing.go and internal/services/invoice_pdf.go
Shiprocket	shipments.go and internal/shiprocket
Dootha	handlers/chat.go
My plants	handlers/my_plants.go and internal/care
Photo ID	handlers/plants*.go and internal/plantnet
Privacy	handlers/privacy.go
Admin map	handlers/dashboard.go
Website data fetching	apps/web/lib/api.ts
Website routes	apps/web/app/
Website cart and auth	apps/web/lib/store/cartStore.ts and authStore.ts
Mobile data fetching	growman-mobile/lib/api.ts
Mobile routes	growman-mobile/app/
Mobile token	growman-mobile/store/authStore.ts
Admin home	apps/admin/app/page.tsx
Admin login gate	apps/admin/components/AdminAuthGate.tsx
This narrative, shorter	docs/COMPLETE-GUIDE.md in both repositories

Figure 16. Open the route table, the model file, and the matching screen. That is enough to start any change.

A closing summary you can speak

Growman is a nursery shop and a care companion. Customers use a Next.js site on Cloudflare and an Expo app. Staff use a Next.js dashboard. All three trust one Go API. The cart lives on the device. The order, the price, the tax, and the stock live in Postgres, and stock is reserved with a conditional update so two people cannot buy the last plant. Razorpay confirms payment twice, from the browser and from a webhook, and only the first confirmation commits. Shiprocket delivers. Invoices follow the Indian financial year. Dootha and my-plants keep the relationship going after the sale. Privacy export and erasure sit next to a one-year retention floor. The design is one process on purpose. The next changes, if the shop grows, are a read replica, a single worker leader, and real migrations, not a split into services.

DOCUMENT

Prepared as a final-year project and interview reference for Growman. It describes the website in apps/web, the admin dashboard in apps/admin, the API in apps/go-backend, and the Expo application in the growman-mobile repository. Public site: growman.live. October 2026.